

Benchmark legacy project

1. Frontend: React + TypeScript vs Alternatives

Criterion	React + TS + Vite (Target)	Vue 3 + TS + Vite	Svelte 5 (SvelteKit)
Speed & Bundle	~16ms • Vite HMR < 50ms • Bundle ~45 KB	~12ms • Vite HMR < 40ms • Bundle ~35 KB	~6ms • Native compilation • Bundle ~18 KB
Kanban & DND Rendering	Optimized components via @dnd-kit and TanStack Virtual.	Fine-grained reactive proxy, good drag & drop rendering.	Direct DOM updates via signals.
Ecosystem	Industry standard: React Router, comprehensive UI ecosystem.	Unified ecosystem (vue-router).	Smaller third-party ecosystem for advanced DND.

Target Choice: Svelte is faster, but we choose **React** because we need to evolve the existing ecosystem rather than replace it entirely, and it is more than enough for our needs.

2. HTTP Backend API: Node.js 24 (Express 5) vs Alternatives

Criterion	Node.js 24 + Express 5 (Target)	Fastify (Node.js 24)	NestJS (Node.js 24)
Speed & Throughput	~15k-20k RPS • HTTP latency ~3-5ms (native async)	> 35k RPS • HTTP latency ~1-2ms	~10k-14k RPS • HTTP latency ~5-8ms
Async & Error Handling	Native support for async route handlers (no wrapper needed).	Native promise support and built-in JSON Schema validation.	Modular architecture with interceptors / exception filters.
Complexity & Testing	Lightweight and battle-tested • Instant testing with Supertest.	Plugin-based architecture • Solid test tooling.	Heavyweight (complex dependency injection pattern).

Target Choice: Fastify is faster, but we choose **Express 5** because the legacy project is already on a Node base; we want to upgrade the existing stack rather than rewrite it, and it easily handles our target load.

3. Database: PostgreSQL 18 vs Alternatives

Criterion	PostgreSQL 18 (Target)	MySQL 8.4 LTS	MongoDB 8
Speed & Queries	< 1-2 ms per indexed query • 25k+ QPS	< 1-2 ms on simple reads • 24k+ QPS	< 1 ms on single document reads
Relational Model	Ideal for Users / Projects / Tasks / Members relations.	Strict relational (less flexible on JSON types).	Non-relational document model (expensive manual joins).
Guarantees & Indexing	Strict ACID transactions, B-Tree and GIN indexes for search.	ACID compliant via standard InnoDB engine.	Document-level ACID transactions.

Target Choice: The legacy project already runs on SQL, so we choose **PostgreSQL** as a direct upgrade to stay relational, keep data consistency, and meet our needs without a full database paradigm shift.

4. ORM & Data Layer: Prisma ORM 7 vs Alternatives

Criterion	Prisma ORM 7.x (Target)	Drizzle ORM	TypeORM
Execution Speed	~2-3 ms optimized SQL overhead (v7) • 10k-14k RPS	< 1 ms near-zero SQL overhead • 18k-22k RPS	~3-5 ms classic ORM overhead • 8k-10k RPS
Type Safety (DX)	Strict automated generation via declarative schema.	Pure TypeScript inference.	TypeScript decorators.
Migrations & Tooling	Robust prisma migrate + visual Prisma Studio.	Manual raw SQL migrations.	Often fragile migrations on evolving schemas.

Target Choice: Drizzle is faster, but we choose **Prisma** because it is easier to use, generates types automatically, and a 1-2 ms difference does not matter for this project.

5. Message Broker: RabbitMQ vs Alternatives

Criterion	RabbitMQ (Target)	Redis (BullMQ)	Apache Kafka
Speed & Throughput	< 1-2 ms latency • 50k+ messages/sec	< 0.5 ms in pure RAM • 100k+ msg/sec	< 2-4 ms • 500k+ msg/sec (streaming)
Messaging Topology	Dedicated AMQP Broker: Exchanges (Direct, Topic, Fanout, Dead-Letter).	Simple FIFO queues backed by Redis lists.	Partitioned distributed log by topics.
Reliability & Persistence	Guaranteed delivery (strict ACKs, disk persistence).	Good (dependent on Redis snapshot settings).	Maximum (strict immutable temporal retention).

Target Choice: Redis is faster in RAM, but we choose **RabbitMQ** because it is a reliable message broker, simple to set up, and more than enough for our background jobs.



References, Sources & Benchmark Test Suites

- **Frontend (DOM latency & bundle sizes):** [JS Web Framework Benchmark \(krausest\)](#) and [Bundlephobia](#).
- **HTTP Backend (RPS throughput & latency):** [TechEmpower Framework Benchmarks \(Round 22+\)](#) and [Fastify / Autocannon Benchmarks Suite](#).
- **Database (QPS & query latency):** [PostgreSQL pgbench Documentation](#), [Sysbench OLTP](#), and [Yahoo! Cloud Serving Benchmark \(YCSB\)](#).
- **ORM & Data Access (SQL overhead):** [Drizzle ORM Benchmarks Suite](#), [Prisma ORM Engine Repository](#), and [TypeORM Benchmark Comparisons](#).
- **Message Broker (msg/s throughput & ACK latency):** [RabbitMQ PerfTest](#), [OpenMessaging Benchmark Framework](#), and [Redis memtier benchmark](#).